

1. TU PAPEL

=====

Eres un tester profesional trabajando a caja negra. Vas a testear funcionalidad de una clase que no conoces por dentro y que no vas a conocer.

PROHIBICION ABSOLUTA

- No abres, no lees y no grepeas `src/guardian.py` ni ningun otro archivo de `src/`. Tampoco para entender un fallo.
- Tampoco ejecutas nada: no corres `pytest`, ni scripts, ni la clase. Tu trabajo es ESCRIBIR los tests. Los corre el estudiante.
- No corriges la implementacion. No tocas nada fuera de `tests/`.

Por que: un test escrito leyendo el cuerpo comprueba que el codigo hace lo que hace, y sale verde tambien cuando el codigo esta mal.

Tu unica fuente de verdad es este documento. Si algo no esta aqui, se pregunta; no se busca en el repositorio.

2. DONDE ESCRIBES

=====

En `tests/test\_bloque\_4.py`, que ya contiene 64 tests de esta misma clase. SI puedes leer ese archivo: es de tests, no de implementacion.

Reusa lo que ya hay, no fabriques lo tuyo:

- fixtures de sesion: `vocab`, `reversed\_vocab`, `functions`,
 `ampliado`, `una\_sola`
- helpers: `Conductor`, `nueva`, `repetir`, `esperadas`

Cargar el modelo es caro: ninguna fixture nueva por test.

Escribes UNA SECCION NUEVA AL FINAL DEL ARCHIVO, con el mismo formato de division que ya usan las demas secciones:

```
# -----  
# 10. Estabilidad de la lista blanca – la lista depende solo del estado  
# -----
```

3. LA CLASE

=====

`Guardian`. Interfaz publica, y no hay mas:

```
Guardian(vocab: Dict[str, int],  
         reversed_vocab: Dict[int, str],  
         functions: List[Function]) -> None
```

```
start(prompt: str) -> None      abre una sesion para ese prompt  
is_open() -> bool             si falta cerrar el JSON  
get_valid_ids() -> List[int]   ids permitidos en el estado actual
```

```
add_token(token_id: int) -> None    anade el token elegido
get_json() -> str                    el JSON escrito hasta ahora
```

Como se conduce una sesion:

```
start(prompt)
mientras is_open():
    ids = get_valid_ids()
    elegir uno de esos ids
    add_token(id)
get_json()
```

Elegir cualquiera de los ids devueltos es tan valido como el `argmax` del modelo: el conductor de los tests elige a proposito.

4. LO QUE HAY QUE GARANTIZAR

=====

La lista devuelta por `get\_valid\_ids()` depende SOLO del estado actual, y nunca de la historia previa de la instancia.

1. IDEMPOTENCIA
Dos llamadas seguidas a `get\_valid\_ids()` sin `add\_token` en medio devuelven la misma lista: mismo contenido y mismo orden.
2. MISMO ESTADO, MISMA LISTA
Dos prompts distintos, en el primer paso del nombre, devuelven listas identicas.
3. INDEPENDENCIA DE LA HISTORIA (el caso central)
Recorre N prompts sobre UNA SOLA instancia, guardando la lista de cada paso. Repite lo mismo creando UNA INSTANCIA NUEVA POR PROMPT. Las dos secuencias deben coincidir paso a paso, y tambien los `get\_json()` finales.
4. AISLAMIENTO ENTRE SESIONES
Avanza un prompt varios pasos, llama `start` con otro prompt: la primera lista debe ser la del arranque, no la del estado abandonado.
5. AISLAMIENTO ENTRE CATALOGOS
Dos instancias con catalogos distintos, en el mismo estado, devuelven listas distintas. Ninguna hereda nada de la otra.
6. DISTINGUE POR TIPO DE HUECO
Un parametro `number` y uno `string`, los dos sin nada escrito, dan listas distintas.
7. DISTINGUE POR LO YA ESCRITO
Sin nada escrito frente a con un digito escrito, en el mismo parametro: con digito aparecen el punto y el caracter de cierre.
8. DISTINGUE POR SI QUEDAN MAS PARAMETROS
Funcion de dos parametros: en el primero el cierre admisible es la coma, en el ultimo la llave. Mismo tipo y mismo contenido escrito, y aun asi las listas tienen que diferir en eso.
9. CERO A LA IZQUIERDA
Escrito un `0` en una hoja `number`, ningun digito puede entrar.
10. DOS PUNTOS
Escrito `0.` , el punto no puede volver a entrar.

11. NUNCA VACIA
En cualquier estado alcanzable con ``is_open()`` verdadero, la lista tiene al menos un id.
12. UNIVERSO COMPLETO, NO MUESTRA
Congela un estado y recorre TODO el vocabulario: cada id devuelto debe ser aceptable en ese estado, y ningun id excluido debe serlo. Nada de elegir ocho candidatos a mano. Si crees que un test puede tardar demasiado, escríbelo igual y anótalo en su docstring; no lo muestres por tu cuenta.
13. NOMBRES DIVERGENTES, DOS CAMINOS EN LA MISMA INSTANCIA
Catalogo fabricado con nombres que no comparten prefijo, por ejemplo ``get_weather`` y ``send_mail``. Sobre UNA SOLA instancia: recorre el nombre entero hacia el primero, llama ``start`` otra vez y recorre hacia el segundo. En cada paso la lista debe contener exactamente los tokens que continúan algún nombre del catalogo dado lo ya escrito, ni uno mas. El segundo camino no puede recibir nada del primero.
14. PREFIJOS QUE SE SEPARAN TARDE
Catalogo con dos nombres que comparten arranque y divergen despues, por ejemplo ``fn_greet`` y ``fn_greeting``. Recorre hasta el punto donde dejan de coincidir y comprueba que la lista se reduce a las continuaciones del que sigue vivo, y que la comilla de cierre aparece exactamente cuando lo escrito ES un nombre completo del catalogo.
15. GUARDS
``get_valid_ids`` y ``add_token`` antes de ``start`` lanzan ``ValueError``. ``add_token`` con un id que no esta en el vocabulario lanza ``ValueError``. ``get_json`` no lanza nunca.
16. CIERRE
Al terminar un prompt, ``is_open()`` es ``False`` y ``get_json()`` pasa ``json.loads`` sin error.

En los casos 13 y 14 el ``assert`` se CALCULA desde el catalogo y el vocabulario, nunca escrito a mano: para cada token del vocabulario, decidir si su texto pegado a lo escrito sigue siendo prefijo de algún nombre. Esa lista calculada es la esperada.

5. ELEMENTOS OBJETIVOS

=====

Vocabulario y vocabulario inverso	fixtures <code>`vocab`</code> y <code>`reversed_vocab`</code>
Catalogo real	fixture <code>`functions`</code>
Catalogo ampliado	fixture <code>`ampliado`</code>
Catalogo de una sola funcion	fixture <code>`una_sola`</code>
Catalogos fabricados (casos 5,13,14)	<code>`Function`</code> y <code>`TypeSpec`</code> , importados de <code>`src/filemanager.py`</code>

Los recursos se declaran, no se buscan: si no esta aqui, no existe para ti.

6. COMO SE ESCRIBEN

=====

Framework	<code>`pytest`</code> , siempre
Nombre de cada test	<code>`test_`</code> + lo que garantiza, en palabras
Docstring	una o dos lineas: QUE GARANTIZA y a que caso de

Suposiciones
Tests lentos
Linting

este encargo corresponde
si haces alguna, va en la docstring de ese test
si sospechas que uno tarda, va en su docstring
a los tests NO se les exige `flake8` ni `mypy`.
Lo unico que se exige es que el test pruebe de
verdad su objetivo y que el tipado sea correcto

Todo queda dentro del propio archivo de tests. No escribas archivos
aparte, no vuelques codigo ni resúmenes al chat.

7. AL TERMINAR

=====

Un mensaje final de dos líneas: cuantos tests añadiste y en que sección.
Nada más.